Password Strength Checker

A Modern Desktop Security Analysis Tool

Name: **Mauli Patel**
Enrollment No.: **125020806030 11**
Subject: **Python Programming Project**

Real-Time Password Security Testing

- **Interactive Assessment:** Evaluates password input immediately during the active typing process using system keystroke event loops.
- **Visual Security Metrics:** Converts password architectural metrics into clear, color-coded components natively inside the GUI canvas.
- **Dynamic Requirement Tracking:** Provides real-time feedback updates across complex safety validation rule dictionaries.
- **Zero External Dependencies:** Designed exclusively for standalone local desktop usage without cloud database, network sync, or login steps.

Focused, Reliable Credential Auditing

- **Input Obscuration System:** Leverages reliable entry character masking symbols (*) paired with a dynamic visual toggle handler.
- **Multi-Tiered Strength Grading:** Classifies user passwords using an explicit four-layer evaluation loop Matrix: Weak, Medium, Strong, and Very Strong.
- **Regex Pattern Verification:** Runs high-efficiency validation strings to verify multi-case characters, integers, and unique punctuation arrays.
- **Length Constraint Multipliers:** Enforces distinct array boundary counts, ensuring short predictable words cannot achieve maximum safety marks.

Integrated Technologies & Framework Matrix

Python Standard Engine

Application Lifecycle

Handles memory configurations, structural class instantiation, synchronous application event routing, and scoring variables setup.

CustomTkinter UI

Graphical Architecture

Supplies scalable anti-aliased desktop layouts, custom canvas dimensions, rounded frames, and automatic operating system theme syncing.

Regular Expressions (re)

Pattern Processing Engine

Performs fast character class scanning across the text buffer arrays in O(N) execution time boundary limits.

High-Speed Input Evaluation Pipeline

01

Capture Buffer

Intercepts raw keystroke operations instantly on key release events.

02

Filter Boundaries

Catches empty strings to restore default application indicators.

03

Regex Verification

Runs structural tests matching lowercase, uppercase, digits, and symbols.

04

Score Aggregation

Loops systematically through results to compile successful metric flags.

05

UI Transformation

Updates indicator heights, hex color shades, and requirement tokens dynamically.

Clean Layout Keeps Metrics Visible

- **App Window Canvas:** Locked to an explicit 450x520 pixel size layout to maintain perfect element proportions across platforms.
- **Horizontal Entry Box Array:** Houses the security-masked entry widget alongside the custom visibility action button component.
- **Normalized Progress Indicator:** A progress tracking bar reading 0.0 to 1.0 decimals that automatically changes hue to track risk grades.
- **Dynamic Requirement Layout:** Displays active complexity targets utilizing explicit red failure (❌) and green completion (✓) indicators.

Modular Structural Architecture

Primary System Methods

- **__init__() constructor:** Configures core application layout, builds container framing, and sets up static validation dictionary keys.
- **toggle_password_visibility():** Handles character masking states, instantly altering string layouts between masked and visible text styles.
- **evaluate_password():** The central entry engine that handles input text gathering, regular expression evaluation, and real-time interface rendering.

Asynchronous Event Hooking Pattern

```
# Asynchronous Event Trigger
self.password_entry.bind("<KeyRelease>",
self.evaluate_password)

# Structural Input String Evaluation Logic
checks = {
    "length": len(pwd) >= 8,
    "upper": bool(re.search(r"[A-Z]", pwd)),
    "digit": bool(re.search(r"\d", pwd))
}
```

Strategic Metric Allocation Matrix

- **Weak Security Profile (Score ≤ 2):** Updates display elements to a bright red hue (#e74c3c) and sets the metric loader tracking bar fill level strictly to 0.25.
- **Medium Security Profile (Score 3 or 4):** Updates display elements to a distinct orange warning color (#f39c12) and sets the metric tracker fill to 0.50.
- **Strong Security Profile (Score 5, Length ≤ 11):** Passes every regex filter check but fails the character extension rule. Employs emerald green highlights (#2ecc71) at 0.75 fill.
- **Very Strong Profile (Score 5, Length ≥ 12):** Requires fulfilling all structural rules alongside an extended length profile. Unlocks the maximum safety state with a distinct teal color layout (#1abc9c) at a 1.00 full rating.

Behavioral Testing Assertions

- **Empty Input Buffer:** Interface components clear calculations immediately, and array string indices reset trackers to standard failure states.
- **Partial Metric Checklist Passes:** Correctly flips matched rule elements to green success checkmarks while maintaining a 'Weak' status for failed rules.
- **Dictionary Pattern Defenses:** Prevents basic short entries from maxing out validation bars if the total character length falls below target rules.
- **Maximum Complexity Verification:** Triggers total validation confirmation, shifting the application context seamlessly into the maximum teal strength configuration.

Project Conclusion & Future Expansions

Project Summary Conclusions

• Successfully implements a responsive local password auditing utility using an object-oriented Python architecture.

• CustomTkinter delivers clean desktop layout aesthetics, while regular expressions supply reliable execution times.

• Ensures user credential safety by operating in a closed, database-free desktop loop layout.

Future Technical Scope

• Integrate advanced offline mathematical entropy analyzers to compute accurate crack-time estimates against GPU hardware arrays.

• Hook application structures securely into breach API repositories (e.g., HaveIBeenPwned) to intercept previously compromised credentials.

• Introduce custom organizational security policy schema configurations.